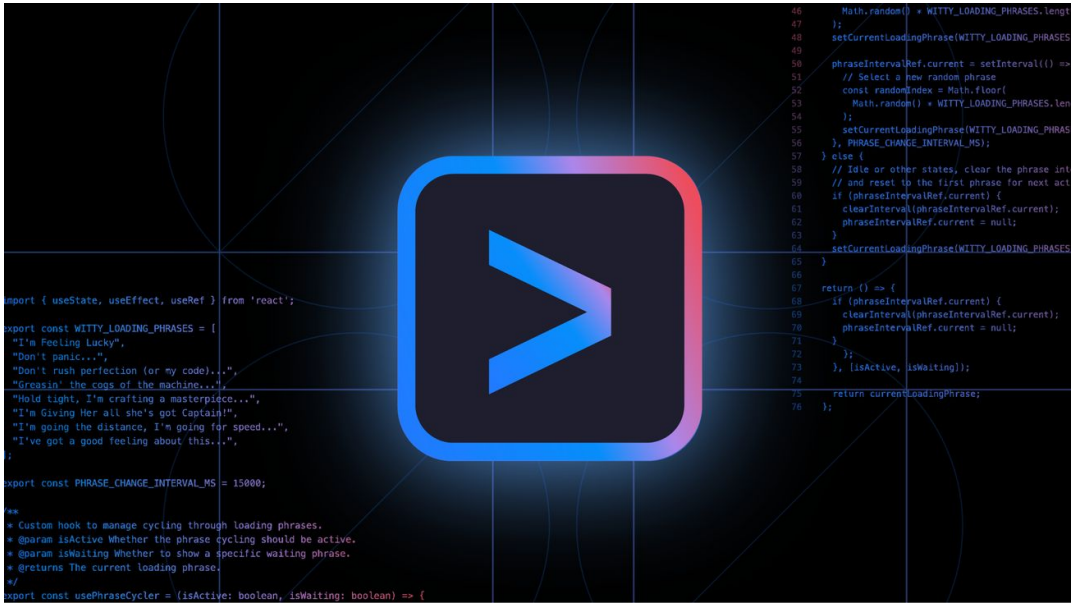


Gemini CLI: Concrete Architecture

Group 7
CMPE 322 Software Architecture
March 13, 2026

YouTube Link:
https://youtu.be/QrLRm8KMfbc

Contribution Breakdown

Omar - Presentation, Abstract, Intro and Derivation Process, High-Level Concrete Architecture, Use Cases, Reflexion Analysis, Lessons Learned, AI Report, Reviewed final draft

Keven - Presentation, Conceptual system architecture

Rowan - Subsystem architecture (both conceptual and concrete), SciTools Understand work

Dylan - Reviewed final draft

Muhammad - Reviewed High-Level Concrete Architecture and Intro and Derivation Process

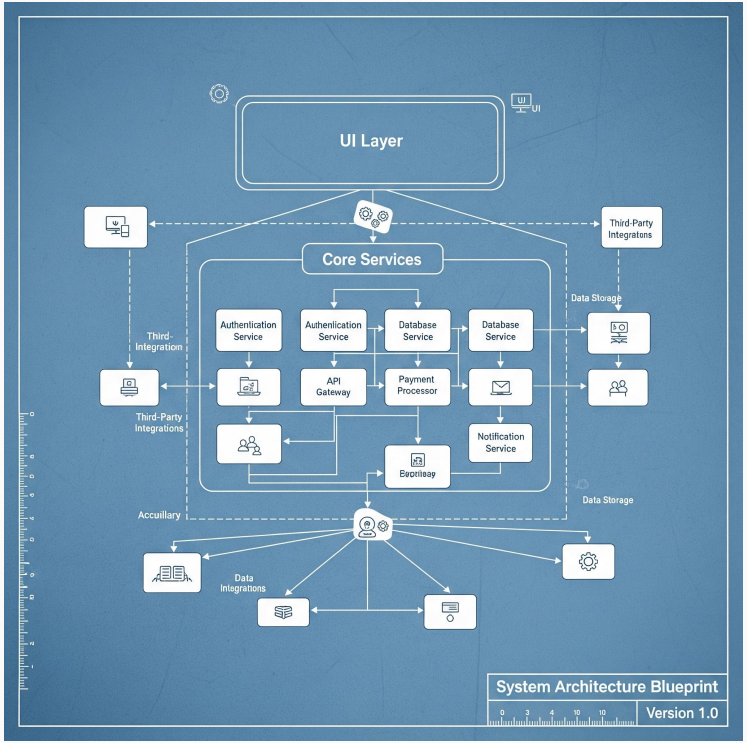
Zonghan Li - **Did not contribute**

System Overview

- Terminal-based AI assistant
- Executes tools (file operations, shell, etc.)
- Uses Gemini API
- Supports external integrations

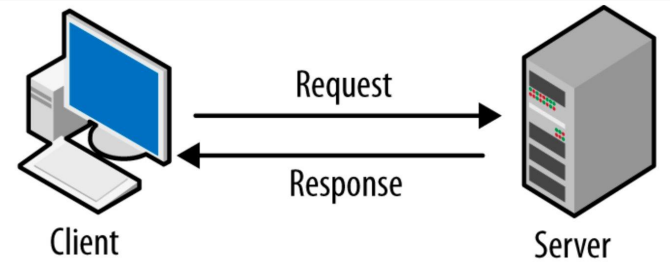
Architecture Derivation Process

- Package boundary analysis
- Import graph tracing
- Responsibility grouping
- Directory structure analysis

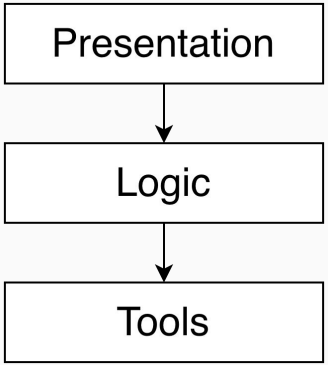


Considered Alternatives

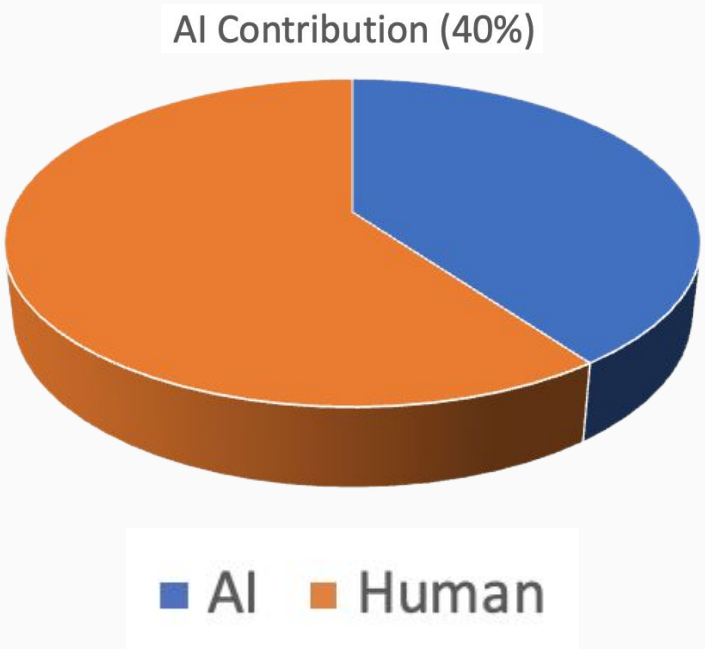
- Alternative 1: Client–Server Architecture
  - Why it was rejected
    - Adds network latency
    - Gemini CLI requires real-time streaming
    - Direct module imports provide better type safety



- Alternative 2: Strict Layered Architecture
  - Why it was rejected
    - The UI must directly access:
      - tool execution state
      - confirmation dialogs
      - MCP server status
    - Routing everything through orchestration would slow down the UI.



AI Collaboration Overview



- Used a dual-AI approach with Gemini Web and a Local Agent.
- The Prompting Protocol involved Two-Tiered Meta-Prompting and Chunking.
- Quality control included manual source code and IDE verification.
- The AI tools contributed an estimated forty percent to the final deliverable.

Conceptual Architecture

Rational for Updating Compared to A1

- Clearer layer separation
- Added missing subsystems
- Defined external boundaries

Application layer

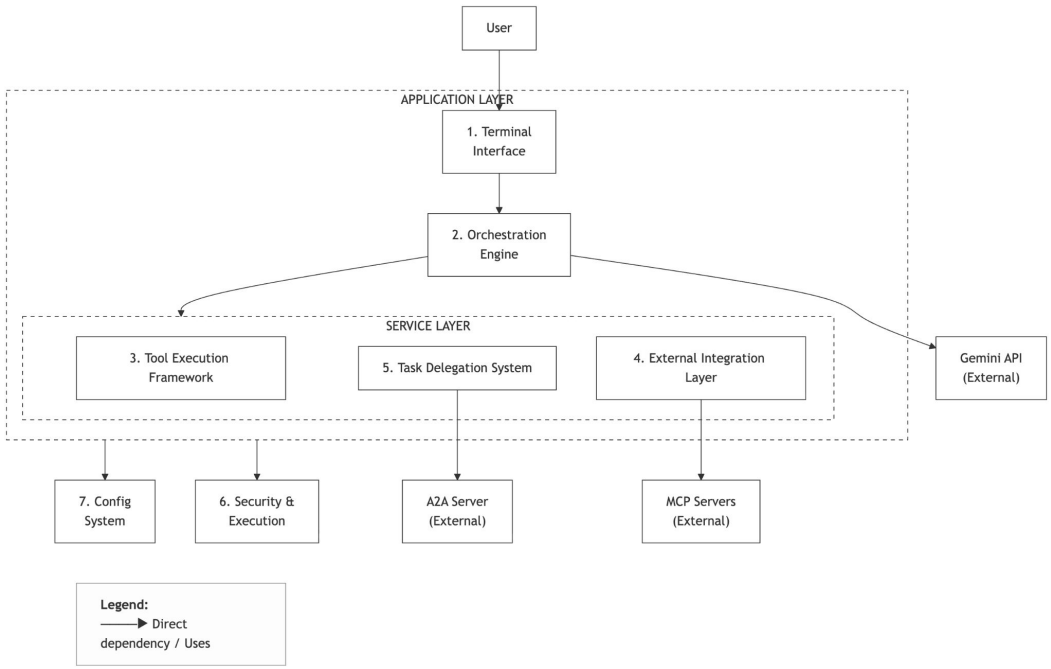
- Terminal Interface
- Orchestration Engine

Service layer

- Tool Execution Framework
- Task Delegation
- External Integration

Cross-cutting

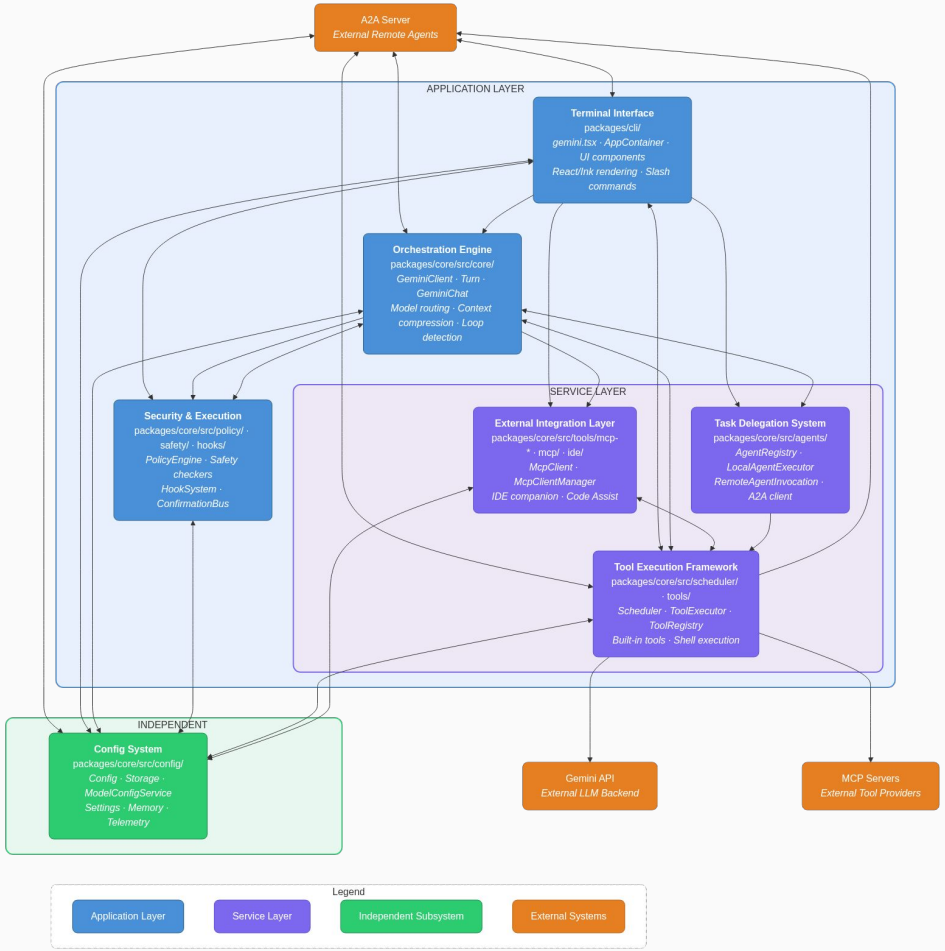
- Security
- Config



Concrete Architecture

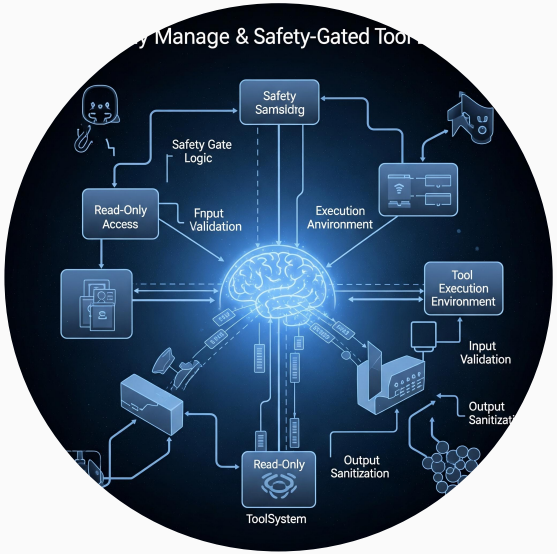
Components

- Terminal Interface
- Orchestration Engine
- Tool Execution Framework
- External Integration Layer
- Task Delegation System
- Security & Execution
- Config System



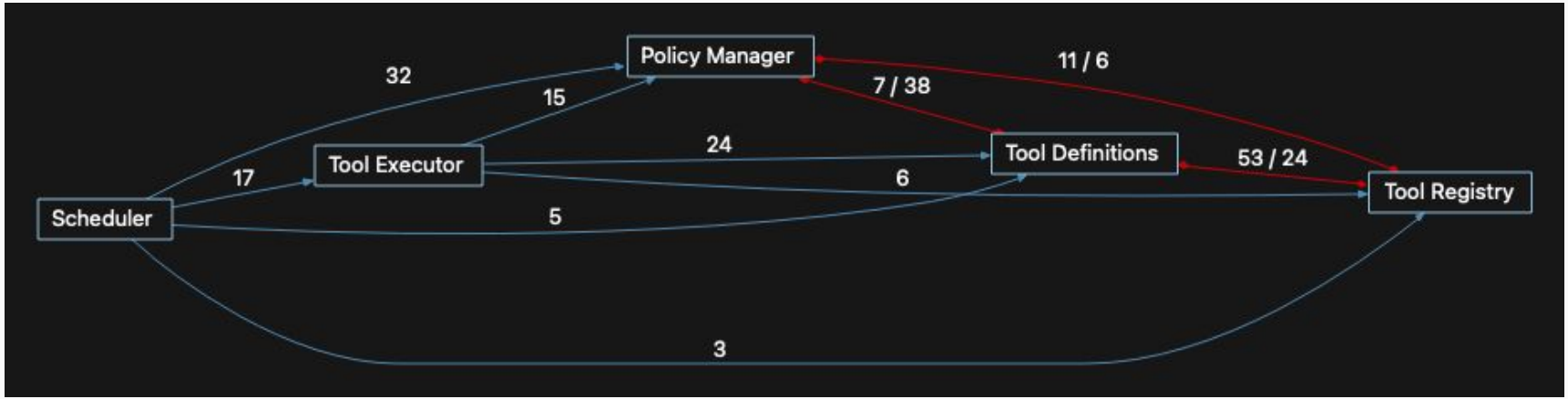
Key Dependencies

1. Terminal Interface ↔ Orchestration Engine (the heaviest, 1300 imports)
2. Orchestration Engine ↔ Tool Execution Framework
3. Task Delegation System → Tool Execution Framework
4. Terminal Interface → Service Layer
5. External Integration Layer →MCP Server

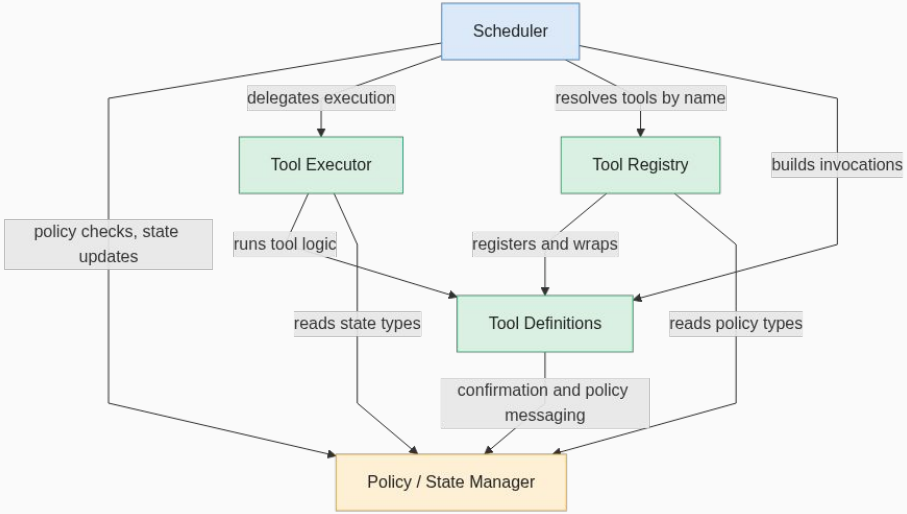


Subsystem Architecture: Tool Execution Framework

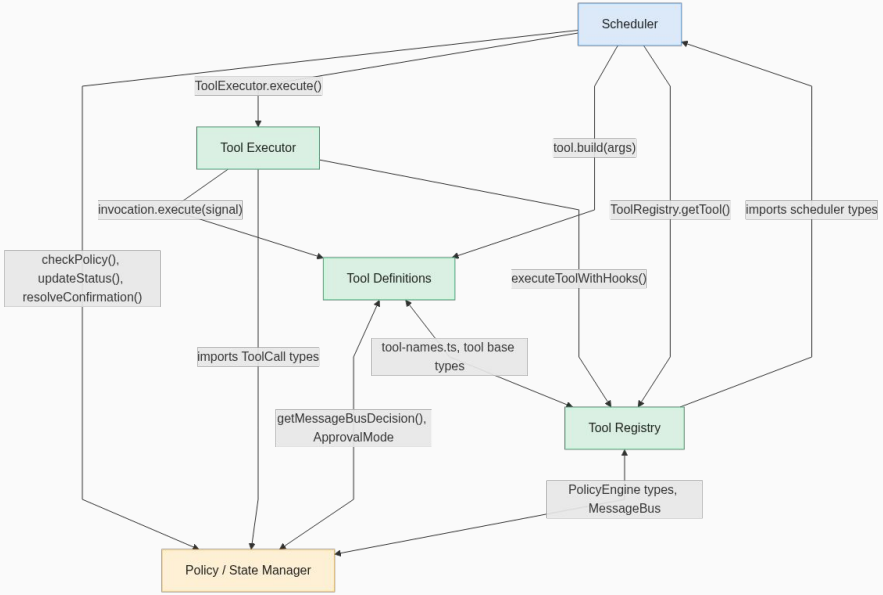
Scheduler: the central manager and coordinator of life cycles of tool calls in the system
Tool Executor: execute tool calls and handles tool logics
Tool Registry: the central directory of all available tools for the system to use
Tool Definitions: defines the base structure and the function for each tool
Policy / State Manager: a subsystem enforces system-wide security and lifecycle rules
State Machine: the scheduler operates on a state machine with some status states
Hub and spoke pattern: the scheduler coordinates and depends on the rest of the system



Subsystem Architecture: Tool Execution Framework Conceptual and Concrete View

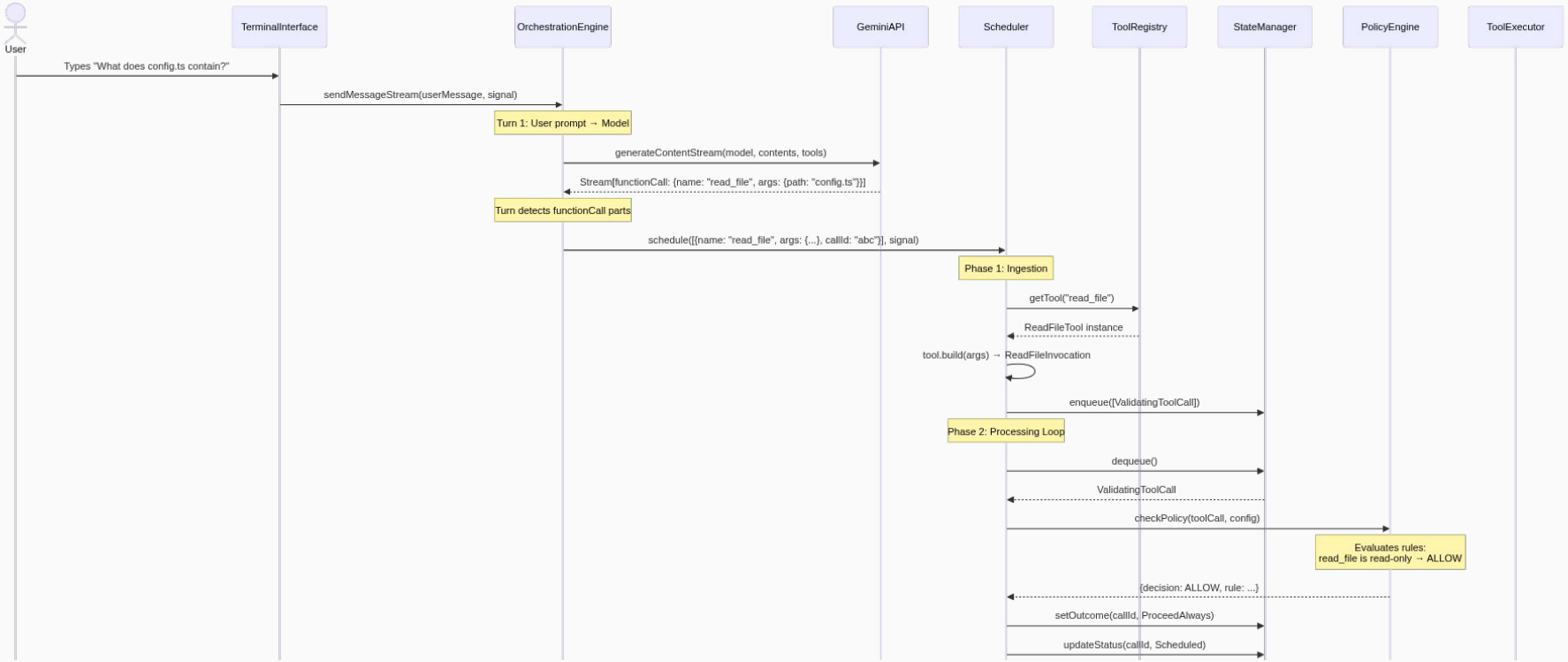


Conceptual View

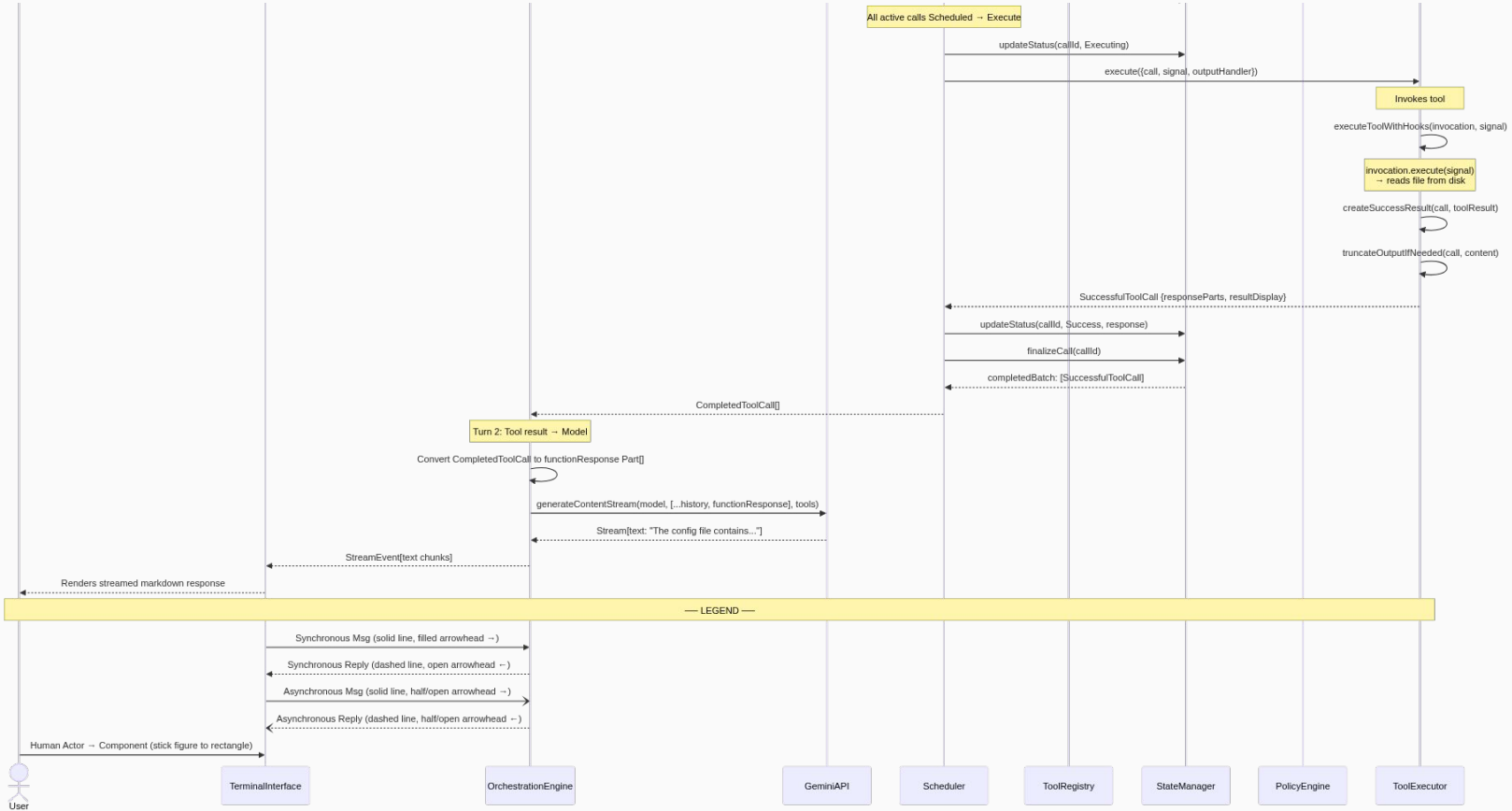


Concrete View

Sequence Diagram Example 1

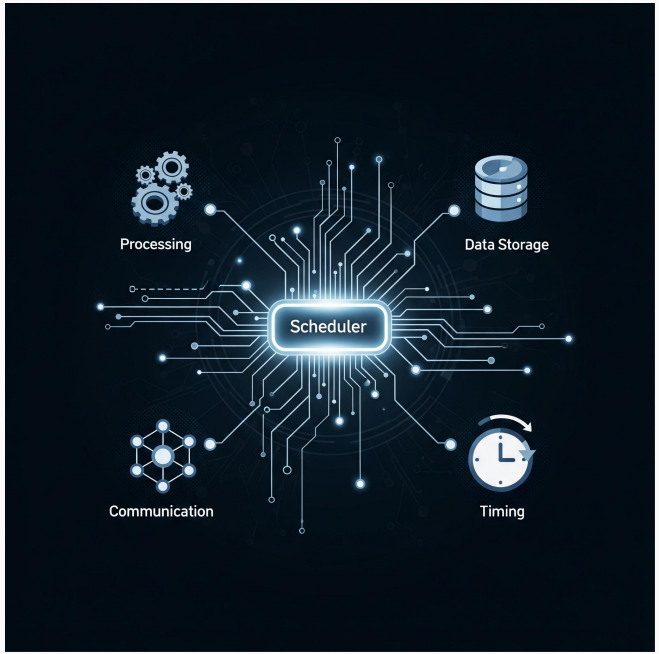


Sequence Diagram Example 1 Con’t



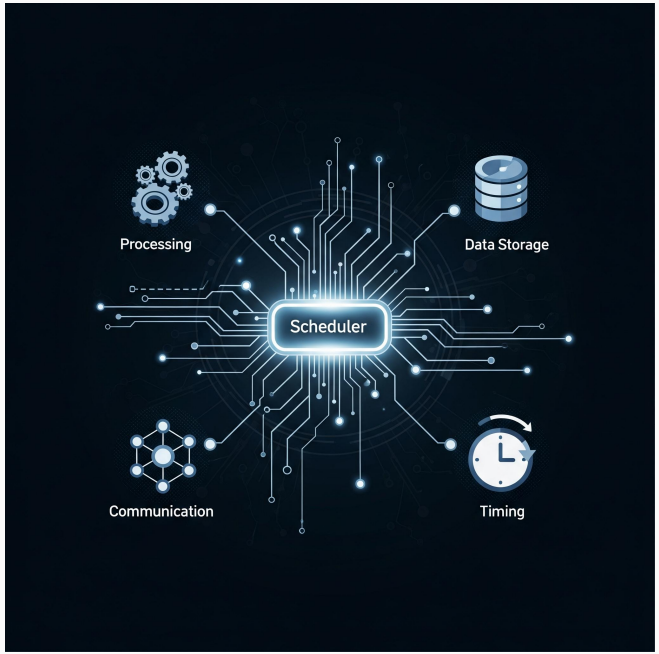
Reflexion Analysis: High-Level

- Layered Monorepo architecture rather client and server (a single-process application)
- Layer bypass instead of strict layered architecture (engine is bypassed)
- Security subsystem is actually a major part of the policy engine
- Orchestration engine and the tool execution framework are actually bidirectional dependent



Reflexion Analysis: Subsystem

- Engine and tools are tightly coupled with bidirectional dependencies through two bridge files
- Boundaries like task delegation subsystem and A2A server is confirmed correct
- **Hub and spoke pattern:** the scheduler coordinates and depends on the rest of the system
- Tool Executor is stateless execution unit: only interacts through execute()
- Registry pattern allows: dynamic tool discovery, MCP tool integration



Lessons Learned & Limits

Lessons

- Real systems often have a diverging concrete architecture than the conceptual one
- Static dependency analysis often reveal more hidden relationships
- Security is a mandatory part of AI system, and must be embedded deep into the system (defense in depth)
- Coordinate tools requires a centralized scheduling

Limits

- Some dynamic runtime dependencies might be missed in static analysis
- Some architectural decisions are inferred from the code structure
- The AI could not understand the whole system due to context window limit

Conclusion

- Layered architecture with centralized orchestration engine
- The system is event-driven
- A hub to spoke style in the tool execution framework
- The concrete architectural analysis reveals tighter coupling and stronger security integration